HIGH

start_up() Bootstrap Replay Equivocates on Every Validator Restart

Repo: ton-blockchain/ton **Branch:** testnet **Commit:** `3bb6abc` **Scope:** Primary — validator/consensus/simplex/consensus.cpp **Date:** 2026-03-14

Summary

The `ConsensusImpl` constructor's bootstrap replay code applies the same missing guard as the `alarm()` race described in a companion finding — but with a more severe trigger: it fires **deterministically on every validator restart**, not just under a race condition. Any validator that crashes or is restarted after having cast a `NotarizeVote` in an unfinished leader window will equivocate by immediately broadcasting a `SkipVote` for the same slot on startup.

Vulnerable Code

During startup, `ConsensusImpl` loads previously cast votes from the persistent database into in-memory state (lines 47–80), then broadcasts `SkipVote` for all slots in the current unannouncedleader window (lines 82–91):

```
// consensus.cpp — start_up(), lines 47–80: restore voted_notar from DB
auto notar_fn = [&](const NotarizeVote& notar_vote) {
  slot->state->voted_notar = notar_vote.id;   // voted_notar restored here
};
std::visit(td::overloaded(notar_fn, final_fn, skip_fn), vote.vote);

// consensus.cpp — start_up(), lines 82–91: broadcast SkipVotes for unannouncedwindow
if (auto window = bus.first_nonannounced_window) {
  auto start_slot = (window - 1) * slots_per_leader_window_;
  auto end_slot   = window       * slots_per_leader_window_;
  for (td::uint32 i = start_slot; i < end_slot; ++i) {
    auto slot = state_->slot_at(i);
    if (slot.has_value() && !slot->state->voted_final) {  // BUG: must also check
!voted_notar
      slot->state->voted_skip = true;
      owning_bus().publish<BroadcastVote>(SkipVote{i}).start().detach();
    }
  }
}
```

The guard at line 87 checks only `!voted_final`. It does not check `!voted_notar`. Because `voted_notar` was restored from the database in the block above, any slot in the window for which the validator previously cast a `NotarizeVote` will receive a `SkipVote` immediately on restart.

Trigger Scenario

1. Validator is the leader for window W (slots `W×k .. (W+1)×k − 1`).
2. A candidate arrives and passes validation → validator broadcasts `NotarizeVote{slot X}` and persists it to the database.
3. Window W has not yet advanced (no `LeaderWindowObserved`) — `first_nonannounced_window = W` in DB.
4. Validator crashes or is restarted (planned maintenance, OOM kill, network partition).
5. **On restart:** bootstrap loads `voted_notar = slot X` from DB. Then the window loop fires and broadcasts `SkipVote{slot X}` because `voted_final = false`.
6. Peers still hold the pre-crash `NotarizeVote{slot X}` in their pools. The validator has equivocated.

> **Why this is deterministic:** Unlike the runtime race condition in `alarm()` (which depends on validation timing relative to the alarm interval), this bug fires unconditionally on every restart where the above state is present. A validator that experiences any crash — OOM, segfault, planned restart — while holding an uncommitted `NotarizeVote` will equivocate on every subsequent startup attempt until the window either finalises or is fully skipped by other validators.

Relationship to alarm() Bug

This is the same root cause as the `alarm()` guard bug: both code paths broadcast `SkipVote` with only a `!voted_final` check, missing `!voted_notar`. There are three independent locations in `consensus.cpp` where this guard needs to be applied:

1. `alarm()` — timer-driven, runtime race
2. `start_up()` window broadcast — happens on every restart (this finding)
3. `try_notarize()` — coroutine resume, runtime race (companion finding)

Impact

A validator restart — even a routine one — silently equivocates if it had a pending `NotarizeVote` in its window. Because the equivocation is not detected by `check_invariants()` (missing case: `NotarizeVote + SkipVote`), no misbehavior report is filed. The validator's weight is counted toward both `notarize_weight` and `skip_weight` for the same slot, distorting certificate formation. The affected slot's candidate block is abandoned.

In practice, validators in production networks restart regularly (software upgrades, infrastructure maintenance, hardware failures). This bug means that under Simplex consensus, routine restarts carry a consensus safety cost that is invisible to the network.

Fix

Add `!voted_notar` to the broadcast guard in `start_up()`:

```
// consensus.cpp — start_up(), line 87 fix
if (slot.has_value()
    && !slot->state->voted_final
    && !slot->state->voted_notar) {    // new guard
  slot->state->voted_skip = true;
```

```
  owning_bus().publish<BroadcastVote>(SkipVote{i}).start().detach();
}
```

The same fix must be applied to all three locations. See the companion `alarm()` finding for the complete fix set.

TON Blockchain Contest Round 2 — Primary scope finding. Submitted by TesseraeVentures.